

Latent Perception-to-Reasoning Interfaces for ECG Triage

Phase 1 technical report: from single-event adapter to multi-event reasoning

Author: Alif Tasbir

Date: 2 October 2026 (updated with the filtered-text baseline, a second sender architecture and serving measurements)

Status: post-viva extension of the MSc dissertation, working towards journal submission

1. Executive summary.....	2
2. Starting point: the dissertation	3
3. How Phase 1 was run	3
4. Weaknesses in the original evaluation and how Phase 1 addressed them	3
5. What changed in the system	4
5.1 Perception: RR-timing branch	5
5.2 Multi-event adapter.....	5
5.3 Recipe r4: hard negatives and side inputs (final).....	5
5.4 Schema-constrained decoding.....	6
6. Efficiency results	6
6.1 Serving conditions: batching and prefix caching (Deviation 23)	7
7. Accuracy results	8
7.1 Primary test (pre-registered, Deviation 16).....	8
7.2 Primary comparison against default and calibrated text.....	8
7.3 Why text fails with many events	9
7.4 False alarms: identified in r3, reduced in r4	10
7.5 Against true annotations	11
7.6 Filtered text baseline (Deviation 20)	11
8. Auditability.....	12
9. Replication on a second sender (Deviation 22).....	13
10. Status of the research questions	14
11. Limitations and threats to validity.....	14
12. Proposed next steps	15
Appendix A. Pre-registered deviations	16

1. Executive summary

The dissertation built and evaluated a small learned adapter that passes an ECG model's internal representation directly into a frozen language model (Gemma 4 E4B) as a few virtual tokens, instead of a JSON text interface. Phase 1 strengthened the evaluation under a pre-registered, multi-seed protocol, identified where the single-event design is limited, and iterated the system from one heartbeat per call to many heartbeats per call, which is where the latent interface's advantage turns out to lie. The main results:

- **Weaknesses identified in the single-event evaluation.** The dissertation compared against the most verbose text interface, measured latency without fixing output length, and reported one training run. With a compact text baseline, fixed-length timing and several seeds, a single event gives the adapter little room: no prompt-processing saving (about -2%), and lower accuracy than text (0.79 vs 0.97 balanced accuracy; the adapter's single-event accuracy varies across seeds, 78.8–95.0%). This motivated the multi-event iteration.
- **The advantage appears when many events share one prompt.** A trained multi-event adapter keeps prompt processing flat (about 183 ms from 1 to 50 events), while text grows with every event: -24% / -50% / -78% time to first token at 10 / 20 / 50 events, and -4% / -14% / -36% time to the urgency decision.
- **Accuracy at scale (pre-registered, 1,195 test windows, 22 held-out patients, 3 seeds, patient-level confidence intervals).** The final adapter (recipe r4) reaches 0.81 / 0.83 / 0.83 / 0.84 balanced accuracy at 5 / 10 / 20 / 50 events. It is more accurate than default text at every N (+0.08 to +0.35) and, the stricter comparison, **more accurate than text with a calibrated threshold at 10 and 20 events (+0.10 each, confidence intervals exclude zero)**, and non-inferior at 5 and 50.
- **How the adapter reached this.** The previous recipe (r3) matched calibrated text but did not beat it, and raised false alarms on about 11% of routine windows. Two targeted changes in r4 (training on hard routine examples, and passing heart rate, RR interval and run length alongside each event) cut false alarms to 3.6% and raised accuracy at every N. The headline is now **equal or better accuracy than calibrated text at 19–78% lower prompt-processing cost**.
- **Auditability holds per event.** Each event's label, tier and urgency flag can be decoded from its slot of virtual tokens as well as from the encoder vector itself, at any of 50 positions. With r4, heart rate and RR interval (R^2 about 0.7) and the presence of a run of abnormal beats (0.96 balanced accuracy) are now also recoverable from the tokens; the previous design could not carry them.
- **A stronger text baseline changes the accuracy picture.** A filtered text prompt that lists only the abnormal beats (normal beats given as a count) is 11–17 points more accurate than the adapter at 5–50 events, raises no false alarms and is equally fast for a single request. It costs 5–19% more prompt tokens, and its length grows by about 68 tokens per abnormal beat. Against the best text prompt, the adapter's case therefore rests on a cost that does not depend on the number of abnormal beats and on carrying information about every beat, not on accuracy.
- **The method replicates on a second, purely convolutional sender (ResNet1D-RR, no recurrence).** Text again collapses as events are added while the adapter stays at 0.77–0.82; the adapter is more accurate than calibrated text at 10 events and non-inferior at 20 and 50; heart rate and RR interval are recoverable from every token slot (R^2 about 0.9). Filtered text is again more accurate at 5–20 events; at 50 events it drops to 0.81 and the difference is inconclusive.
- **Under realistic serving the adapter is the most efficient interface, including against filtered text.** With requests batched, it serves 10–30% more decisions per second than filtered text at 12–28% less energy per decision, and it fits every batch size tested on a 12

GB GPU; text listing every event cannot be batched at 50 events, and filtered text fits only some batches of 8 because its length depends on the content.

- **Remaining weaknesses:** the adapter still raises false alarms on 1–4% of routine windows where text raises none; filtered text is more accurate when the question is known in advance; the two r4 changes were made together, so their separate effects are not yet known; text remains better for a single event.

2. Starting point: the dissertation

The pipeline paired a CNN-LSTM beat classifier (MIT-BIH, inter-patient DS1/DS2 split) with Gemma 4 E4B (4-bit, frozen). Arm A sent each beat's HealthEvent JSON as text; Arm B projected the classifier's 32-d context vector into 4 virtual tokens. The task was the urgency tier (routine / priority / urgent) of one beat, defined by a fixed escalation rule on the classifier's output.

Measure (80-event set)	Arm A (JSON)	Arm B (adapter)
Mean prompt tokens	645.2	486.0 (–24.7%)
Mean generation latency	9,892 ms	8,574 ms (–13.3%)
Task accuracy	100%	95.0% (seeds: 85.0% ± 8.7 pp)
Class recoverability	100% by construction	67.6% (probe)

Table 1. Dissertation headline results (dissertation Table 4.1).

Research questions: RQ1, does the adapter reduce token count, latency and memory; RQ2, does it preserve task accuracy; RQ3, what does a vector interface cost in auditability, and can that cost be measured.

3. How Phase 1 was run

- **Pre-registration.** Every change of design was written into the analysis plan as a numbered deviation before any data it affected (19 deviations, each with its reason). Confirmatory and exploratory analyses are labelled as such.
- **Fair baselines.** Besides the dissertation's full JSON (A-full), a compact JSON (A-compact: label, confidence, run length, SQL) and a label-only arm were added; A-compact is the strongest text baseline and is used throughout.
- **Seeds and statistics.** At least 3 training seeds per recipe; paired bootstrap confidence intervals; for the final test, a record-level (patient-cluster) bootstrap, because windows from one patient are correlated.
- **Held-out patients.** Adapters train on DS1 (19 records) with 3 held-out DS1 records for validation; all headline results are on DS2 (22 records never used for training or tuning).
- **Reproducibility.** Resumable, bit-identical training checkpoints (unit-tested; one r4 run was paused and resumed mid-training without loss); atomic result files; 66 automated tests pass.

4. Weaknesses in the original evaluation and how Phase 1 addressed them

The dissertation delivered a working pipeline and a first evaluation. Reviewing it for journal standards showed five weaknesses in how the single-event design was evaluated. Each one led to a specific change in Phase 1.

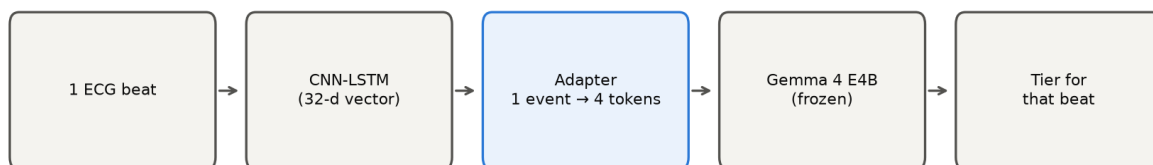
Weakness	Why it matters	What Phase 1 did	What it showed
One comparison baseline (full JSON, the most verbose text form)	A saving against a verbose prompt may not hold against a well-designed one	Added compact JSON and label-only text arms	Against compact JSON, a single event leaves little to save: interface tokens 4 vs ~57, but end-to-end prompt length similar (486 vs 458)
Latency measured with free-length answers	Answer length, not the interface, can drive total time	Timed every arm with output length fixed	With equal output length, arms differ by $\leq 3\%$; decode speed is identical (~169 ms/token), so latency gains must come from the prompt side
One training run reported as the headline	Adapter training is sensitive to the random seed	Multiple seeds per configuration, reported as mean and range	Single-event accuracy varies across seeds (78.8–95.0%)
Methods text describes 3 full-batch updates	Reproducibility	Checked against the code	The code performs 192 per-example updates (64×3 epochs); the description is updated accordingly
Auditability measured only as class recoverability	Needs to locate where information is lost	Added inverse decoders and an RR-timing encoder	Recoverability reproduced (69.6%); the loss sits in the encoder, and the RR branch raises it to 81% (mean of 10 seeds)

Table 2. Weaknesses identified in the single-event evaluation and the Phase 1 response to each.

Taken together, these showed that a single heartbeat per call gives a latent interface little room to help: the fixed prompt scaffold dominates the cost, and text already carries the one event well. The interface's natural advantage is when many events must share one context, where text grows by about 60 tokens per event and the adapter by 4. Phase 1 therefore iterated the design towards multi-event reasoning (Sections 5-8).

5. What changed in the system

Dissertation (single event)



Phase 1 (multi-event)

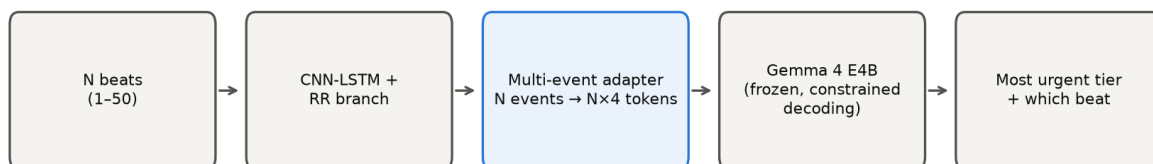


Figure 1. Pipeline before (single event) and after Phase 1 (multi-event). The highlighted block is the component trained in this work.

5.1 Perception: RR-timing branch

A 5-feature RR-interval branch (pre-RR, post-RR, local mean RR and two ratios) was added to the CNN-LSTM's context stage. Over 10 training seeds (Deviation 19), it improves on the reference encoder on every seed for overall accuracy, normal, S and V beats, and class recoverability from the 32-d vector; it does not reliably detect F beats (6 of 10 seeds above the reference's 0.3%). Cost: one beat of latency (the post-RR interval).

Measure (DS2)	Reference CNN-LSTM	RR encoder, 10 seeds: mean $\pm$ SD [range]	Seed 0 (used downstream)
Accuracy	0.854	0.926 $\pm$ 0.019 [0.889, 0.949]	0.911
S sensitivity	8%	48% $\pm$ 21% [15%, 73%]	38%
V sensitivity	77%	93% $\pm$ 3% [86%, 97%]	97%
F sensitivity	0.3%	5% $\pm$ 7% [0%, 18%]	0%
Class recoverability (probe)	69.6%	81.2% $\pm$ 5.0% [74.6%, 88.3%]	84.6%

Table 3. RR encoder across 10 seeds (Deviation 19). Seed 0, on which all adapters were trained, was the only seed when it was chosen and ranks 3rd, 4th and 7th of 10 on accuracy, S sensitivity and recoverability (representative by the pre-registered rule); it has the highest V sensitivity of the 10.

5.2 Multi-event adapter

- N context vectors $\rightarrow$ N $\times$ 4 virtual tokens: a per-event linear projection, learned event-position embeddings (up to 50 slots), and per-token L2 normalisation with a learnable scale (this removed the norm drift that broke generation at large N).
- Task: the most urgent tier among N consecutive beats, plus which beat. Deterministic training targets in the same JSON schema.
- Training recipe r3: batch 1, AdamW 5e-4 with warm-up and cosine decay, validation every quarter epoch with early stopping from 2 epochs, 375 class-balanced windows (N = 1, 5, 10, 20, 50). Memory-saving (logits only for target tokens, gradient checkpointing) keeps peak GPU memory at 9.5–10.3 GB on a 12 GB card.
- The first recipe (r1, 8-step gradient accumulation, 111 optimizer steps) under-trained: one seed failed completely (0.38 on its own training windows). Diagnosing and fixing this was pre-registered as Deviation 11.

5.3 Recipe r4: hard negatives and side inputs (final)

Evaluating r3 (Section 7) exposed two weaknesses, each traced to a cause, and r4 (Deviation 18, registered before training) targets both:

- **False alarms on routine windows (about 11%).** They concentrated on windows containing a normal beat classified with low confidence, which the training set almost never showed. r4 adds such routine windows as **hard negatives**: 72 training and 23 validation windows whose lowest beat confidence is below 0.8.
- **Facts the vector does not carry.** Heart rate, RR interval and the length of a run of abnormal beats (one of the two urgent rules) were absent from the adapter's input, while the text arm includes them. r4 passes three scaled **side inputs** with each event (35-d input instead of 32-d), at no extra token cost: still 4 tokens per event.
- Training otherwise as r3, scaled to the larger set (447 training windows, 170 validation windows). Validation is harder than for r3 because it includes the hard negatives, so r3 and r4 validation scores are not directly comparable; the DS2 test set is identical.

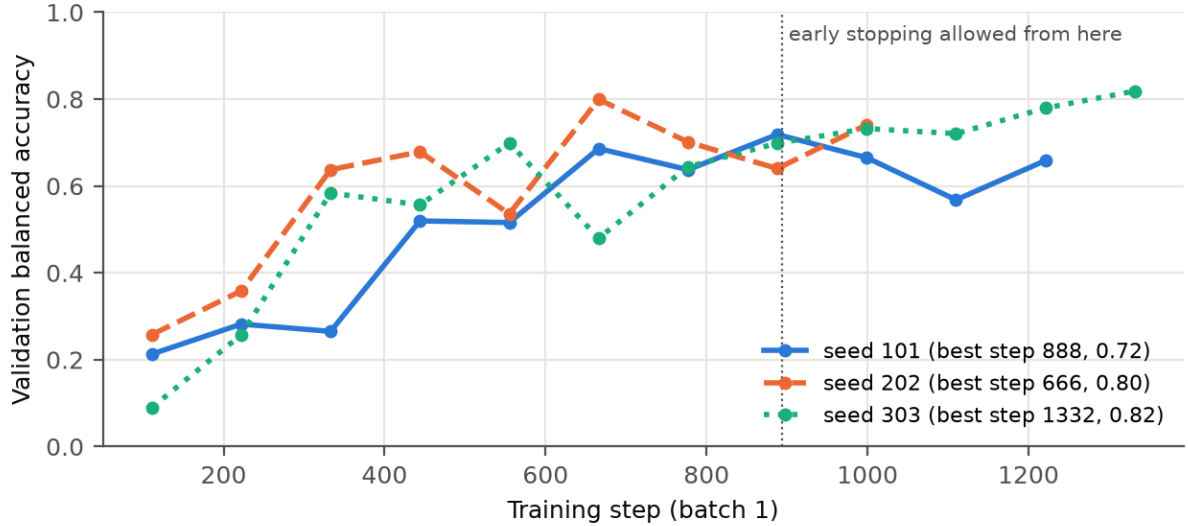


Figure 2. Validation balanced accuracy during training, recipe r4, three seeds. The best checkpoint per seed is selected on validation only; seed 303 was still improving when it reached the 3-epoch cap and was evaluated as pre-registered.

5.4 Schema-constrained decoding

Free generation produced malformed JSON 9–20% of the time (misnamed fields, loops). Every arm now uses the same constrained decoder: the JSON structure is forced, the tier is restricted to the three valid words and chosen by the model's own scores, and free-text fields cannot break the JSON. Parse rate is 100% for every arm; a check confirmed the decoder changes no tier (380/380 windows identical before and after its last correction).

6. Efficiency results

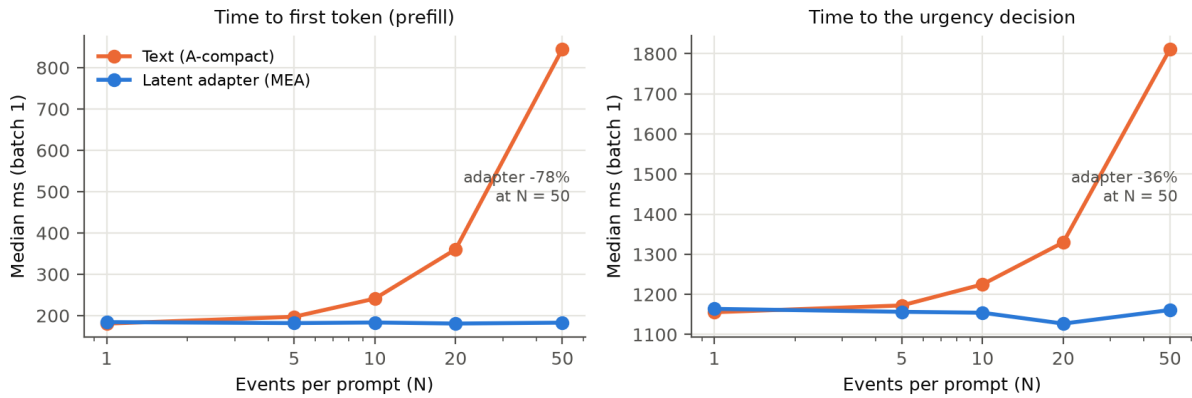


Figure 3. Median latency per request (batch 1, 21 DS2 windows per N). Left: time to first token. Right: time until the urgency tier is generated (token 7 of the constrained answer).

Events per prompt	Prefill vs text	Time to decision vs text	Other
1	0%	+1%	no advantage
5	-8%	-1%	no advantage
10	-24%	-4%	-79 MB context memory
20	-50%	-14%	-177 MB
50	-78% (845 → 183 ms)	-36% (1,812 → 1,161 ms)	-460 MB; energy 391 → 230 J

Table 4. Cost of the latent interface relative to compact text (E3, E3b, Deviation 14). End-to-end response time is dominated by generating the answer (~160 ms/token for every arm), so full-response latency depends on answer length, not on the interface. Recipe r4 keeps 4 tokens per event (the side inputs enter the same projection), so these costs apply to it unchanged.

6.1 Serving conditions: batching and prefix caching (Deviation 23)

The figures above are single requests with nothing reused. A serving system batches requests and reuses the computation for the shared start of the prompt (prefix caching). Both were measured on the same windows (batch sizes 1, 4 and 8; 3 repeats; medians), on the same 12 GB GPU, with the framework capped at 90% of the card's memory so that a batch that does not fit is recorded as such instead of being silently moved into system memory, which would measure paging rather than the model (Deviation 23a).

N	Batch	Adapter	Text, all events	Filtered text
10	1	0.98 /s · 83 J	0.92 /s · 111 J	0.97 /s · 95 J
10	4	3.19 /s · 39 J	2.23 /s · 65 J	2.80 /s · 49 J
10	8	4.72 /s · 33 J	does not fit	3.63 /s · 43 J (3 of 6 batches fit)
20	4	3.17 /s · 39 J	1.70 /s · 92 J	2.49 /s · 54 J
20	8	4.55 /s · 33 J	does not fit	4.00 /s · 38 J (3 of 6 fit)
50	1	1.00 /s · 89 J	0.60 /s · 232 J	0.96 /s · 92 J
50	4	2.79 /s · 48 J	does not fit	2.25 /s · 63 J (12 of 15 fit)
50	8	4.11 /s · 38 J	does not fit	3.72 /s · 43 J (3 of 6 fit)

Table 5. Decisions per second and energy per decision (GPU, joules) when requests are batched (adapter: r4 seed 101). 'Does not fit': the batch exceeds the memory cap of the 12 GB card. Filtered-text figures at batch 8 come only from the batches that fit, which are its shorter prompts, so they favour filtered text.

- **Against text with every event listed:** prefill 34–81% faster and 25–62% less energy per decision at N = 10–50 for single requests; with batching, text cannot be served at all at N = 50 or at batch 8, while the adapter fits every configuration (peak 10.0 GB).
- **Against filtered text:** equal for single requests (within about 5%), but ahead once requests are batched: 10–30% more decisions per second and 12–28% less energy per decision. Filtered text fits only some batch-8 batches at every N, because its length depends on how many abnormal beats a window contains.
- **Prefix caching** shortens every arm's prefill and does not change the ranking.
- **What this means for an agent pipeline:** the latent channel has a fixed memory and compute footprint per event, so batch sizes can be planned in advance; a text interface's footprint depends on what the sender has to say.

7. Accuracy results

7.1 Primary test (pre-registered, Deviation 16)

Test set: 1,195 DS2 windows (22 patients), balanced by tier and by the class of the most urgent beat, including 400 natural-prevalence windows for false alarms. Primary statistic: adapter (3-seed mean) minus A-compact balanced accuracy, with a patient-level bootstrap confidence interval; non-inferiority margin -0.05 .

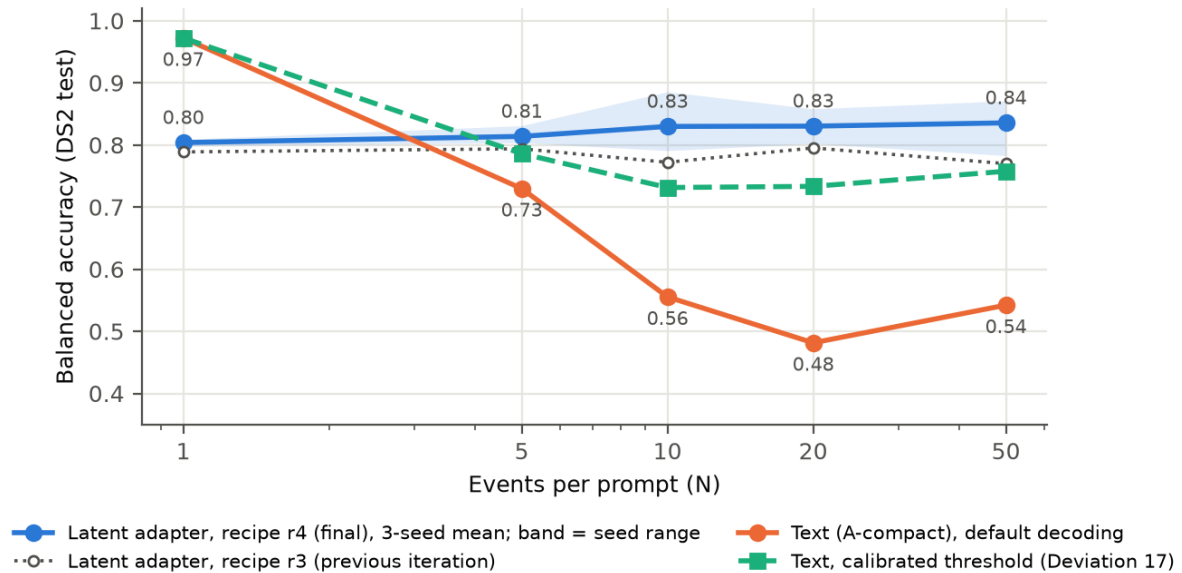


Figure 4. Balanced accuracy on the DS2 test set by window size. The final adapter (r4) stays at 0.80–0.84 for every N and sits above the previous recipe (r3) at every N; default text falls sharply from N = 5; text with a calibrated threshold recovers part of the gap.

N	Windows	Text (default)	Text (calibrated)	Adapter r3	Adapter r4 (final)
1	177	0.97	0.97	0.79	0.80
5	175	0.73	0.79	0.79	0.81
10	169	0.56	0.73	0.77	0.83
20	157	0.48	0.73	0.80	0.83
50	117	0.54	0.76	0.77	0.84

Table 6. Balanced accuracy on the class-balanced DS2 test windows (adapters: mean of 3 seeds). Text answers are identical across the r3 and r4 comparisons (same windows, prompts and decoder).

7.2 Primary comparison against default and calibrated text

The calibrated text arm adds a bias on the routine score at the tier step, chosen on the DS1 validation set only (Deviation 17); it lifts text from 0.48–0.56 to 0.73–0.76 at $N \geq 10$ and is the fair comparison. Against it, r3 was non-inferior but never superior. r4 is superior at $N = 10$ and 20, with patient-level confidence intervals that exclude zero, and non-inferior at $N = 5$ and 50.

N	r4 – default text [95% CI]	r4 – calibrated text [95% CI]	Verdict vs calibrated text	r3 – calibrated text (before)
5	+0.08 [+0.01, +0.16]	+0.03 [-0.04, +0.10]	non-inferior	+0.02 [-0.06, +0.10]
10	+0.27 [+0.20, +0.35]	+0.10 [+0.03, +0.17]	superior	+0.04 [-0.04, +0.13]
20	+0.35 [+0.23, +0.46]	+0.10 [+0.01, +0.18]	superior	+0.06 [-0.03, +0.16]
50	+0.29 [+0.16, +0.42]	+0.08 [-0.02, +0.17]	non-inferior	+0.01 [-0.11, +0.13]

Table 7. Adapter minus text, balanced accuracy, with patient-level (record-cluster) bootstrap 95% confidence intervals; non-inferiority margin -0.05 .

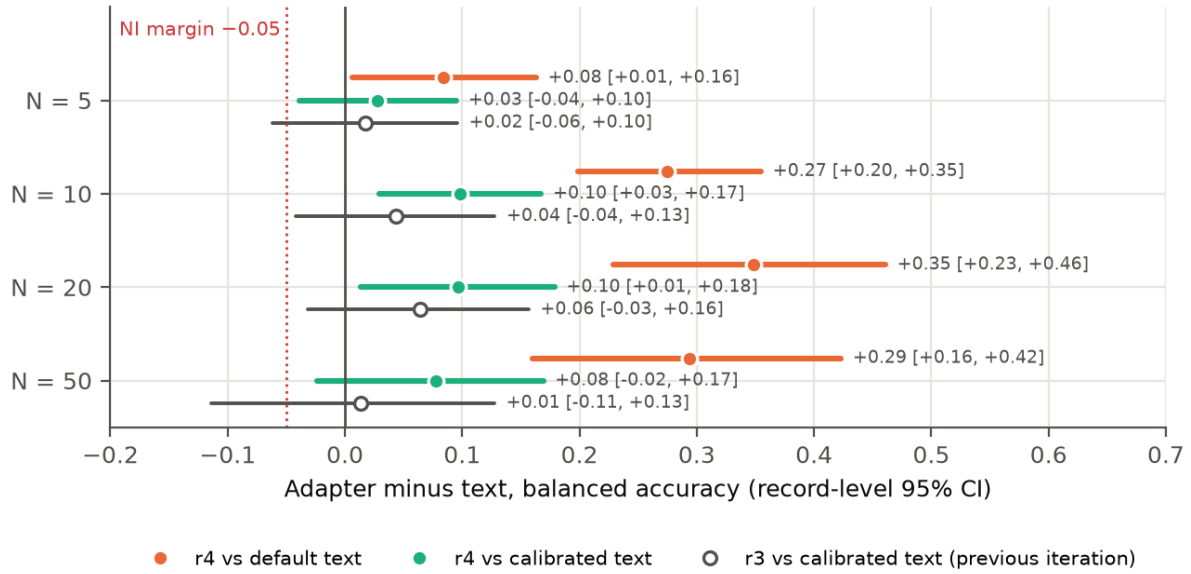


Figure 5. The comparisons of Table 7. Dotted red line: non-inferiority margin. Hollow markers show the previous recipe (r3) against calibrated text.

7.3 Why text fails with many events

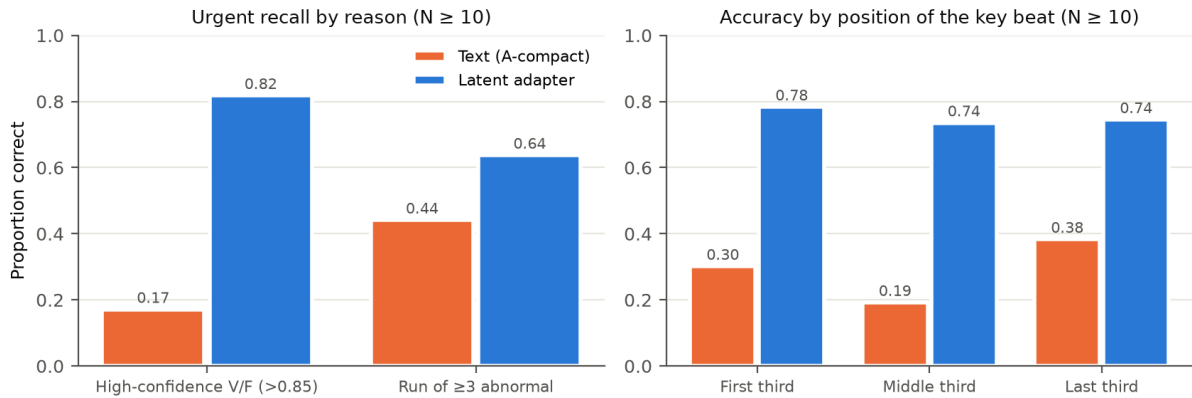


Figure 6. Left: text misses urgent windows caused by a single high-confidence V/F beat (a numeric threshold, 0.85, buried in a long list) far more than runs of abnormal beats. Right: text is worst when the key beat is in the middle of the list, the 'lost in the middle' effect; the adapter (r4, 3 seeds) shows no position effect.

Both mechanisms are documented in the literature: position effects in long contexts (Liu et al., TACL 2024; Hsieh et al., Findings of ACL 2024) and label bias corrected by calibration (Zhao et al., ICML 2021). A fairness point: the text arm is zero-shot, whereas the adapter was trained on this task; a trained text baseline is planned.

Urgent recall on windows made urgent by a run of three or more abnormal beats barely changed between recipes (r3 0.61, r4 0.64, text 0.44), though it varies less across r4 seeds (0.56–0.74 vs 0.32–0.82). The run-length side input therefore helped little with this rule at this evaluation size.

Confusion at N = 50 (row-normalised)

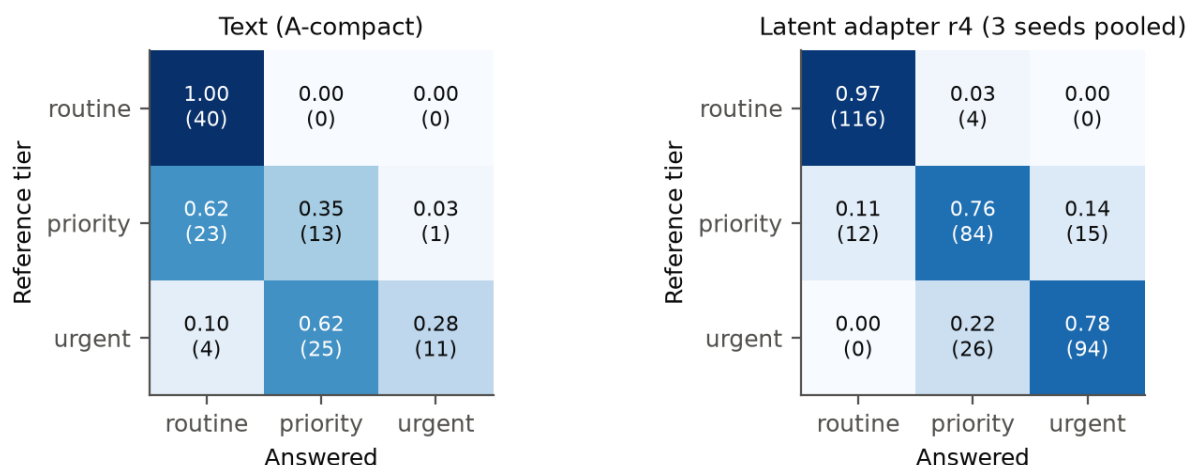


Figure 7. Confusion at N = 50. Text under-escalates (priority → routine, urgent → priority) and never raises a false alarm; the r4 adapter never calls an urgent window routine and escalates 3% of routine windows.

7.4 False alarms: identified in r3, reduced in r4

With r3, the adapter escalated about 11% of routine windows (text: 0%). They sat on windows containing a normal beat classified with low confidence (median lowest confidence 0.56 vs 0.99; an abnormal class as strong second choice), and only 12% of them contained a truly abnormal beat. Threshold calibration could not target them because the validation set had almost no such windows. Training on such windows as hard negatives (r4) cut the rate by two thirds, at every seed.

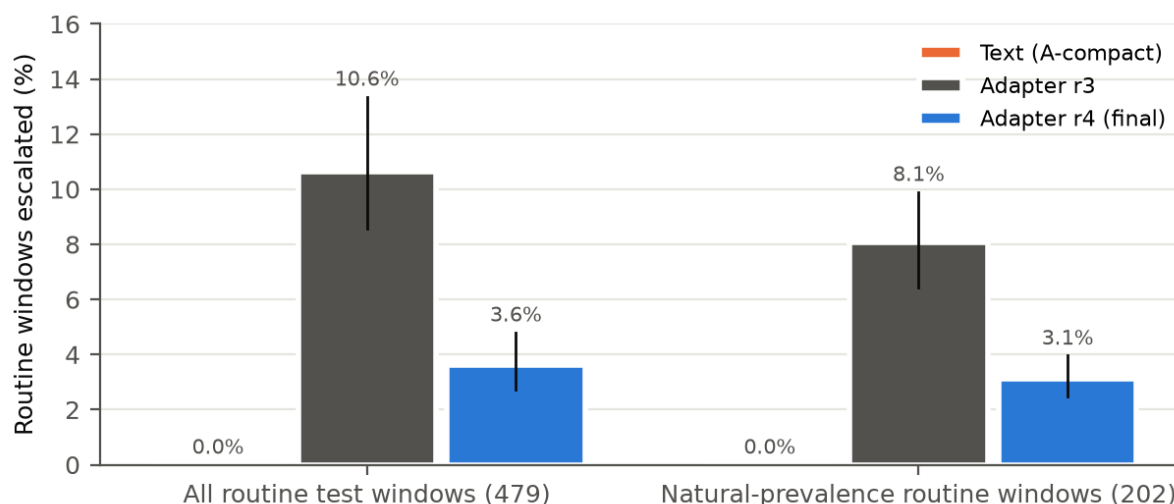


Figure 8. Routine windows escalated to priority or urgent (bars: mean of 3 seeds; lines: seed range). Text never escalates a routine window.

N	Accuracy: text	Accuracy: r4 (3 seeds)	False alarms: text	False alarms: r3	False alarms: r4
5	0.84	0.89–0.90	0%	4.8–8.1%	3.2%
10	0.61	0.87–0.91	0%	7.7–11.5%	1.9–3.8%
20	0.60	0.88–0.93	0%	8.3–10.4%	4.2–6.2%
50	0.56	0.92–0.95	0%	2.5–10.0%	0–2.5%

Table 8. Natural-prevalence windows (100 per N; 40–62 routine windows per N), where the tiers occur at their real frequency.

7.5 Against true annotations

Scoring escalation against the MIT-BIH annotations (does the window contain a truly abnormal beat?) shows that the encoder's own rule has sensitivity 0.92–0.94 but specificity only 0.46–0.52. The clinical accuracy ceiling is therefore set by perception, not by the interface; the adapter (analysed for r3) tracks the reference (sensitivity 0.89–0.98) while default text misses more truly abnormal windows (0.67–0.71 at $N \geq 10$).

7.6 Filtered text baseline (Deviation 20)

The strongest objection to the efficiency claim is that a text prompt need not list every beat. The filtered prompt lists only the non-normal beats (with their position in the window) and gives the normal beats as a count; everything else (scaffold, instructions, constrained decoder) is unchanged. Filtering loses no task information: the filtered window's tier equals the reference on all 1,195 test windows.

N	Adapter r4 (3 seeds)	Filtered text	Adapter – filtered [95% CI]	Prompt tokens: adapter / filtered (median)
1	0.804	0.966	–	516 / 679
5	0.814	0.988	–0.17 [–0.22, –0.13]	532 / 679
10	0.830	0.982	–0.15 [–0.19, –0.11]	553 / 682
20	0.830	0.945	–0.12 [–0.16, –0.07]	593 / 683
50	0.836	0.942	–0.11 [–0.17, –0.05]	713 / 750

Table 9. Adapter against the filtered text prompt (default decoding; calibration changes the filtered arm by about 0.01 or less). Record-cluster bootstrap 95% CIs.

- **Filtered text is more accurate at every N** (confidence intervals exclude zero) and raises no false alarms (adapter 2.7–4.8%). For single requests, time to first token and time to the decision differ by at most 1.5% (about 180 ms for both); with batching the adapter is ahead (Section 6.1).
- **Its cost depends on the content.** Each listed abnormal beat adds about 68 tokens, so at 50 events the filtered prompt is longer than the adapter's in half of the windows (those with more than about 1.5 abnormal beats). The adapter's prompt length depends only on N.
- **What filtering gives up.** It relies on perception's labels to decide what to drop, and the receiver (and an auditor) gets no information about the normal beats. The adapter passes per-event information for every beat (Section 8).
- The pre-registered decision rule for this comparison anticipated a filtered prompt that is cheaper but less accurate; applied literally it returns 'advantage holds' because the filtered prompt is longer. We do not read it that way: against filtered text, the latent interface offers no accuracy or latency advantage on this task.

8. Auditability

- **Adapter tokens lose nothing that the vector holds.** Decoders trained on DS1 and tested on DS2 recover predicted label (99.5%) and tier (95%) from single-event tokens as well as from the 32-d input.
- **Per-event traceability survives multi-event use.** For every slot tested (0, 9, 19, 49 of a 50-event window) label, tier and urgency flag decode at the level of the input vector (Figure 9). Caveat: decoders are slot-specific (a slot-0 decoder applied at slot 49 drops to 0.64–0.91), so an auditor needs a position-aware decoder.
- **A gap found in r3, closed in r4.** The encoder's 32-d vector does not carry heart rate, RR interval or run length ($R^2 \leq 0.3$), so r3's tokens could not either; these are exactly the fields the JSON adds. With r4's side inputs they are now in the tokens: heart rate R^2 0.59–0.78 and RR interval 0.65–0.77 (input: 0.98 / 0.95), and whether a run of ≥ 3 abnormal beats is present at 0.93–0.98 balanced accuracy (input 0.98; r3 0.56, near chance). Signal quality is still not carried.
- **Carried is not the same as used.** Run length reaches the language model in r4, yet urgent recall on run-based windows rose only from 0.61 to 0.64: the model makes partial use of it. Better training for this rule is a candidate next step.
- **Pre-generation check:** an uncertainty score computed from the vector alone predicts the adapter's wrong answers (AUROC 0.89–0.97) and flags noise and powerline interference (AUROC 0.99).

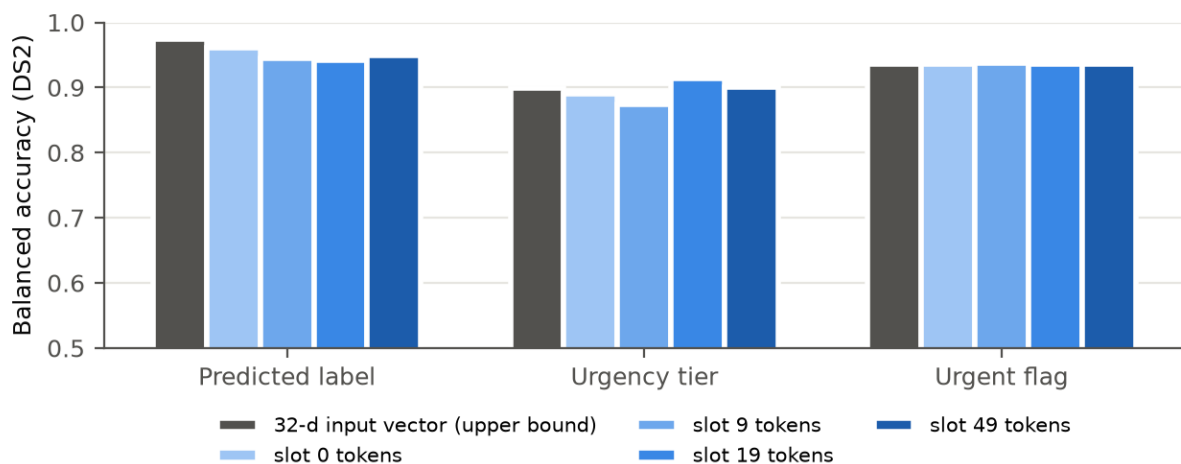


Figure 9. Balanced accuracy of decoding each event's fields from its virtual-token slot (recipe r3, mean of 3 seeds) versus from the encoder's 32-d vector.

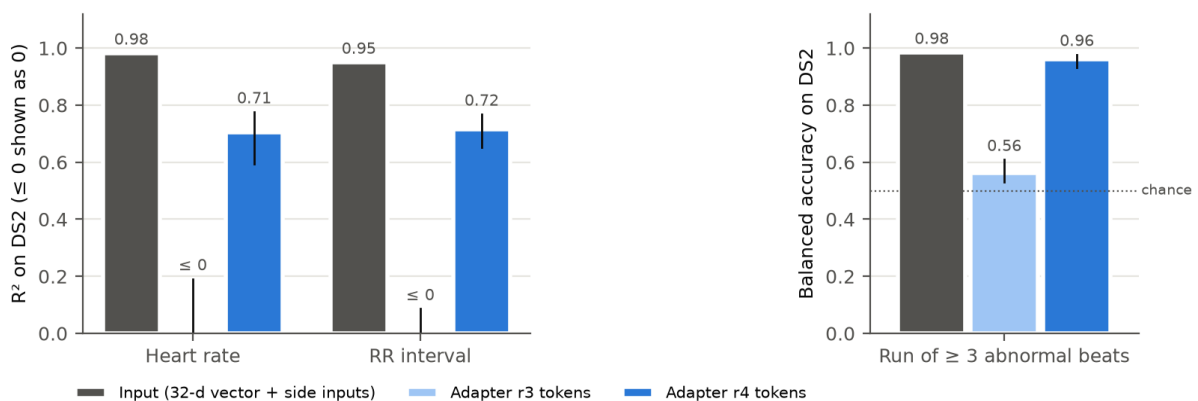


Figure 10. Facts carried per event after the r4 change, decoded from slots 0 and 49 of each seed (bars: mean; lines: range). Left: heart rate and RR interval (R^2). Right: presence of a run of ≥ 3 abnormal beats (balanced accuracy; Deviation 18b, a pre-registered replacement for an R^2 metric that proved ill-posed on this split).

9. Replication on a second sender (Deviation 22)

All results so far use one sender architecture (a CNN-LSTM). To test whether a different kind of non-transformer model can use the same latent channel, a purely convolutional sender was trained (ResNet1D-RR: residual 1-D convolutions, no recurrence, the same RR branch and the same 32-d context vector). Everything downstream followed the frozen r4 protocol unchanged: its own DS2 test windows (1,182; the tiers depend on the sender's predictions), three adapter seeds, both text arms and calibration on its own validation windows. It reached 0.955 on DS1 validation (gate 0.85).

N	Adapter (3 seeds)	Text	Text, calibrated	Filtered text	Adapter – calibrated text	Adapter – filtered
1	0.789	0.964	0.964	0.955	–	–
5	0.770	0.715	0.787	0.959	–0.02 [–0.11, +0.08]	–0.19 [–0.25, –0.13]
10	0.823	0.585	0.732	0.929	+0.09 [+0.02, +0.16]	–0.11 [–0.18, –0.04]
20	0.780	0.586	0.733	0.907	+0.05 [–0.03, +0.13]	–0.13 [–0.20, –0.06]
50	0.794	0.550	0.733	0.800	+0.06 [–0.02, +0.14]	–0.01 [–0.09, +0.08]

Table 10. Second sender: balanced accuracy on its DS2 test windows, with record-cluster bootstrap 95% CIs (non-inferiority margin –0.05). Calibrated filtered text is shown in the last column.

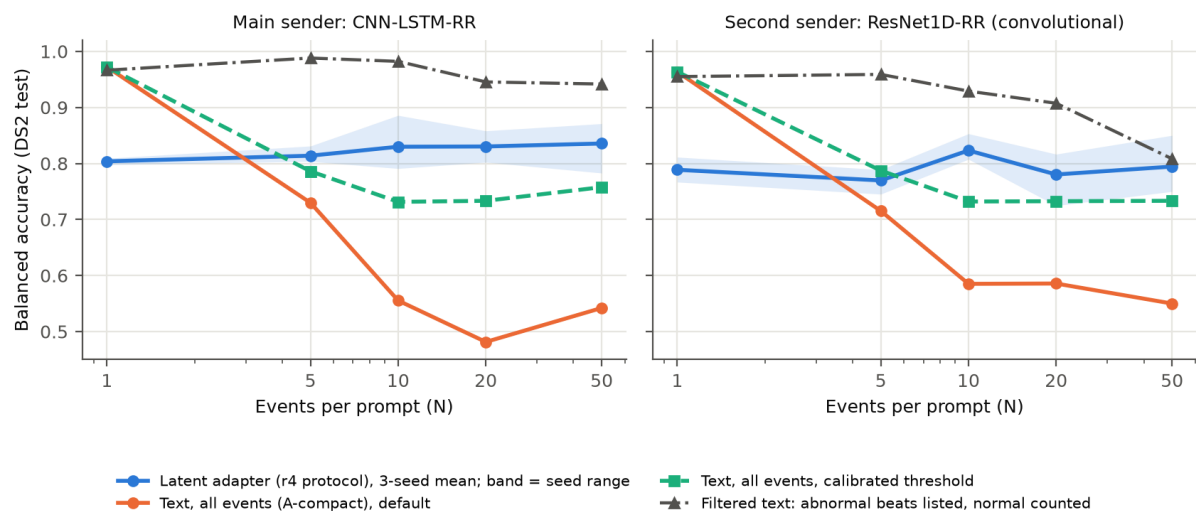


Figure 11. Both senders side by side. The pattern replicates: text with every event listed falls as N grows, the adapter stays roughly flat, and filtered text is the most accurate arm, except at $N = 50$ on the second sender.

- **Accuracy (RQ2).** The adapter is more accurate than calibrated text at 10 events and non-inferior at 20 and 50, but not at 5; on the main sender it was superior at both 10 and 20. False alarms: 1.3–2.1% of routine windows (text 0%). Every arm produced a valid answer on every window.
- **Cost (RQ1).** Time to first token 16–79% lower than text with every event listed at $N = 5$ –50 (713 vs 3,456 prompt tokens at $N = 50$); 6–10% lower than filtered text at most N , with no difference at $N = 20$.
- **Recoverability (RQ3).** From every token slot: beat label 0.91–0.97, tier 0.98–0.99 and run ≥ 3 0.93–0.98 balanced accuracy; heart rate R^2 0.89–0.94 and RR interval R^2 0.89–0.92, higher than on the main sender (0.59–0.78). The sender's own vector is somewhat less class-recoverable (0.76 ± 0.07 vs 0.81 ± 0.05).
- **Why filtered text drops at 50 events here** (exploratory, after the fact). Not prompt length (median 684 tokens vs 750 on the main sender). Almost all errors are urgent windows answered 'priority', and this sender's urgent windows carry sparser, more borderline evidence: often a single urgent beat among about 13 listed abnormal beats, a deciding V/F

confidence just above the 0.85 threshold, or urgency only through a run of three. Filtered text's accuracy therefore depends on how the sender's evidence is distributed; the adapter's does not change much (40 windows per sender, so this is indicative only).

10. Status of the research questions

Question	Dissertation	Now
RQ1 cost	Token and latency savings vs full JSON	Refined: the saving is a multi-event effect; 19–78% lower prefill and 4–36% faster decision at $N = 10$ –50 vs compact text; replicated on a second sender. Against filtered text, single-request latency is similar; with batching the adapter has 10–30% higher throughput, 12–28% less energy per decision and a fixed memory footprint (Section 6.1)
RQ2 accuracy	95% on the evaluation set; seed variance flagged as open	Resolved with seeds and held-out patients: superior to calibrated text at $N = 10$ –20 and non-inferior at 5 and 50 (r4); superior to default text at $N = 5$ –50; false alarms reduced to 3.6% (text 0%); text remains preferable for single events. Filtered text is more accurate at every N ; second sender superior to calibrated text at $N = 10$ and non-inferior at 20 and 50
RQ3 auditability	67.6% recoverability vs 100%	Extended: cost located in the encoder; RR branch raises it to $81\% \pm 5$ (10 seeds); per-event decoding holds at every slot; heart rate, RR and run length now carried in the tokens (r4); replicated on a second sender (heart rate and RR R^2 about 0.9); uncertainty check available

Table 11. Research questions: dissertation answer and how Phase 1 refined or extended it.

11. Limitations and threats to validity

- Many design changes (Deviations 9–19) were made after early results. Each was logged before the data it affected, but the DS2 test set has now informed several iterations. A confirmatory test of the frozen system on an unseen database (INCART) is pre-registered (Deviation 21).
- Recipe r4 changed two things at once (hard negatives and side inputs), so their separate contributions are not identified; an ablation is planned. One r4 seed was still improving at the epoch cap and was evaluated as pre-registered.
- One database so far (MIT-BIH, 22 test patients), one receiving language model (Gemma 4 E4B, 4-bit), one consumer GPU. Two sender architectures have now been tested; a second receiving model is future work.
- The task is communication fidelity: the language model reproduces a fixed rule over perception outputs, which a three-line rule also solves. The contribution is about the interface, not clinical reasoning.
- The text baselines are zero-shot; the adapter is trained. This matches the setting studied (a frozen receiver shared by several agents, not fine-tuned for one sender), but a receiver fine-tuned on text could do better. The filtered text prompt, the strongest text baseline tested, is more accurate than the adapter on this task (Section 7.6).
- False alarms are reduced but not removed (1–4% of routine windows across both senders; text 0%), and single-event accuracy remains below text. Windowing delays alerts by up to the window length and gives one decision per window.
- **Fusion (F) beats are essentially not detected** (sensitivity $5\% \pm 7\%$ over 10 encoder seeds). This is a property of the data split rather than of the model: 96% of the training F

beats come from one patient (record 208) and 93% of the test F beats from another (record 213), and the two patients' fusion beats differ. In training they are premature and resemble ventricular beats (correlation of the mean beat with V 0.73); in the test patient they occur in a regular rhythm and resemble normal beats (correlation with N 0.85, with V 0.12), so the model labels most of them normal and the RR branch has no timing cue to use. F is widely reported as the weakest class under this inter-patient split. For triage, urgent decisions on F beats therefore rest on V detection or run length. Remedies (more patients with F beats, e.g. the INCART database; augmentation; merging F with V for triage) are possible but not needed for the interface claims.

- Encoder seed variation: S sensitivity ranges from 15% to 73% across seeds, and seed 0 (used for every adapter) has the highest V sensitivity of the 10 (97% vs a mean of 93%), which may slightly favour the downstream results. Perception specificity (0.46–0.52) limits clinical accuracy.
- No human evaluation; that needs ethics approval.

12. Proposed next steps

1. **Confirmatory test on INCART** (32 patients, never used for any decision): the system is frozen and versioned first, then run once with fixed hypotheses (Deviation 21).
2. **Compression curve**: 1, 2, 4 and 8 tokens per event, measuring accuracy and recoverability against cost (Deviation 24).
3. **Ablation of recipe r4**: train with hard negatives only, to separate their effect from the side inputs'.
4. **A second receiving language model** is left as future work.
5. Writing: present Phase 1 as a post-viva extension that strengthens the dissertation's evaluation (fair baselines, seeds, held-out patients) and carries the idea to multi-event reasoning; deviations summarised in one table.

Questions: (1) The framing is efficient communication between agents, with ECG triage as the case study; is TMLR or Engineering Applications of Artificial Intelligence the better fit? (2) Is one receiving language model acceptable for submission, with a second as future work? (3) Should a small clinician rating study be planned, given the ethics lead time? (4) How should the updated methods description (192 per-example updates) be recorded alongside the submitted dissertation?

Appendix A. Pre-registered deviations

#	Change	Why
1–4	Training description; seeds; baseline family; forced-length timing	Strengthen the original evaluation
5	Efficiency gate; generation cap 256 tokens	Claim efficiency only where CI excludes 0
6–8	Tier-balanced training; multi-event pilots; stratified pilot	Diagnose single-event limits
9	Multi-event adapter on the RR encoder	Test accuracy where efficiency exists
10	Class-balanced test windows	Test set was dominated by V beats
11	Recipe r2 (batch 1, schedule, larger validation) + early-stop minimum	r1 under-trained
12	Schema-constrained decoding (+4 corrections)	Malformed JSON, not truncation
13	N = 50 (recipe r3)	Largest efficiency gain
14	Time-to-decision latency	Latency beyond prefill
15	Exploratory robustness analyses	Patient clustering, true labels, seed vote
16 / 16a	Enlarged test set (1,195 windows); step 1 retired	Precision; claim moved to multi-event
17	Threshold calibration of both arms	False alarms; fair comparison
18 / 18b	Recipe r4: hard negatives + heart rate, RR and run-length side inputs; run-length decodability measured as run ≥ 3 balanced accuracy	False alarms and facts missing from the vector; the planned R^2 was ill-posed on this split
19	RR encoder retrained with 10 seeds	Its gains were single-run numbers
20	Filtered text baseline (abnormal beats listed, normal beats counted)	Strongest objection to the efficiency claim
21	Confirmatory test of the frozen system on INCART	DS2 has informed several iterations
22	Second sender architecture (ResNet1D-RR)	Results came from one sender architecture
23 / 23a	Serving conditions: batching, prefix caching, energy; memory cap so that oversized batches are recorded, not paged	Timing so far was batch 1, no caching
24	Compression sweep (1, 2, 4, 8 tokens per event)	Cost against accuracy and recoverability
25	Text generation stopped at the tier token, if verified on DS2 first	Compute for the confirmatory run

Table 12. Deviations 1–25 (full text in the analysis plan).